

AI for Science • 科研助手方向

MoreHand 项目介绍文档

意图驱动 • 抗幻觉 • 全流程科研助手 Agent

面向全国高校 AI 大模型挑战赛 AI4S 赛道的参赛作品说明

项目详情与使用方式 • 项目亮点 • AI 测试落地 • 关键技术 • 图示 • 发表与获奖 • 优化说明

文档版本 v1.0 • 2026-10-26

代码仓库 https://github.com/Through1n/ai_research_agent

在线演示 https://ai4s-research-agent.app.workbuddy.host/

目录

执行摘要 1

一、项目详情及使用方式 1

二、项目亮点 7

三、AI 测试技术落地使用情况概述 10

四、项目涉及的关键技术 13

五、项目相关的图片/照片 17

六、项目之前是否发表过或获奖 21

七、若项目是在其他项目基础上优化 22

附录 A · 配套材料索引 24

附录 B · 版本与复核 24

执行摘要

Executive Summary

单元测试	技术文档代码引用	双栏识别	OCR 词级准确率
238/238 全部通过	**236 条** 0 疑似错误	**10/10** 真双栏正文页	**93.9%**召回 98.5%

MoreHand 把「文献解析 → 抗幻觉问答 → 版式阅读 → 检索追踪 → 引用图谱 → 学术写作」串成闭环。与通用 RAG 的差别有两条：一是先保住版式再谈检索，双栏论文先按栏切分再入库；二是把「答错能不能被抓出来」做成代码判定：拒答闸门、角标约束、逐句引用校验、修复环与 References 白名单四道防线跨生成前中后，全部可在真实大模型下复跑验证。

本文所有数字均来自脚本实跑，并附有复跑命令；代码里找不到的功能、没有实测支撑的数字，一律没有写进来。

一、项目详情及使用方式

本节回答三个问题：这是什么、给谁用、怎么用起来，并如实列出当前尚未覆盖的边界。

（一）项目是什么

MoreHand 是一个把「文献解析 → 抗幻觉问答 → 版式阅读 → 检索追踪

→ 引用图谱 → 学术写作」串成闭环的科研助手 Agent。

它不是「通用大模型套一层学术皮肤」。真正的差别在两处：

第一，它先保住版式，再谈检索。通用 RAG 直接按 PDF 文本流切块，双栏论文的左右栏会被搅在一起，检索到的片段本身就是错的。MoreHand 先做版面感知解析（双栏几何判定、扫描件识别、插图与图注关联、标题树、表格与公式抽取），再切块入库。

第二，它把「答错能不能被抓出来」做成代码判定。竞品比「答得好不好」，MoreHand 比「答错了能不能被抓出来」：拒答闸门、角标约束、逐句引用校验、修复环、References 白名单，四道防线跨生成前中后，全部可在真实大模型下复跑验证。

（二）面向的用户与要解决的痛点

科研工作者每天的真实卡点集中在「检索不可信、版面会串读、流程被切碎、图表会丢失、参考文献不可信」五处，通用工具各有盲区。

表 1-1　通用工具的盲区与 MoreHand 的对应做法

痛点	通用工具为什么解决不了	MoreHand 的做法
检索与引用幻觉	通用模型凭记忆答题，页码、年份、整条参考文献都可能被编造；真实标题配错误被引数更隐蔽	每条结论强制带可点击的原文页码锚点；检索不到可靠依据时拒答而不是编造
版面感知缺失	学术 PDF 超半数是双栏，naive 提取按对象流输出，左右栏交叉串读	4 条必要条件同时成立才判双栏；先按栏切分再入库
科研流程割裂	读文献用 PDF 阅读器、管文献用 Zotero、画图用 draw.io、写框架用 Word	一个意图入口统一编排六大模块，每一步都可溯源
图表丢失	PDF 里的图表被当纯文本抽出来，图注与图分离	插图与图注按水平重叠关联，多模态模型解读图表内容
写作参考文献不可信	让模型写参考文献等于让模型编文献	References 由系统从 PDF 真实抽取后确定性拼接，模型没有机会碰

（三）产品形态：九页交互 + 一个智能体内核

界面共 9 个交互页面（导航定义见 MENU@ui/sidebar.py:15），每个页面对应一个真实科研动作。

表 1-2　九个交互页面及其解决的问题

页面	解决的问题	关键能力
智能总助手	我想做 X	意图识别 → 任务规划 → 专家路由 → 工具调用闭环 → Trace 可视化
文献库管理	文献入库与组织	细粒度解析（标题树/表格/插图/公式/参考文献）、内容级语义去重、版本差异
跨文献问答与溯源	论文里到底怎么说的	混合检索 + 逐句引用校验 + 页码级溯源锚点 + 拒答闸门
版式阅读与智能翻译	照原文读懂	双栏原生渲染不串栏、中英对照同步滚动、划词翻译
论文检索与追踪	找什么、追什么	arXiv/PubMed 多源检索、增量追踪、引用含金量标注
引用图谱与综述	这批文献什么关系	PageRank/社区发现/frontier 五维打分、图谱驱动综述、Future Work 过滤
写作辅助与可视化	写成能交的稿子	无幻觉 References、CSV 自动出图、数值事实注入 Caption、中英互译
模型接入与账号	能不能用我自己的 Key	5 家平台 OpenAI 兼容热切换、连通性测试
关于本系统	这项目到底怎么做的	架构说明、创新点、离线降级矩阵

底层是自研 Agent 内核（core/）：规划（planner）、多专家路由（router，6 专家白名单）、函数调用闭环（agent，22 个 Function Schema 全部接入白名单、无孤岛，见 build_registry@tools/definitions.py:379）、三层护栏（guardrails）、记忆（memory）、沙箱（sandbox）与全链路 Trace（trace）。

（四）使用方式：四条上手路径

按评委手上有什么条件，选择下面四条路径之一，四条都指向同一套业务代码。

表 1-3　四条上手路径与适用人群

路径	适用人群	步骤	耗时
① 在线直接体验	评审、评委	打开在线演示地址 → 侧栏「一键加载演示数据（6 篇）」 → 进「跨文献问答与溯源」提问	3 分钟，无需注册、无需 Key
② 本地一键起	想看代码的评审	git clone → 双击 start.bat（Windows）或 bash start.sh（macOS、Linux） → 浏览器自动打开	5 分钟内
③ Docker 部署	部署验证	docker compose up -d → docker compose ps 期望 Up (healthy) → docker compose logs --tail=80 ai4s-agent	3 分钟
④ 桌面版	无开发环境的评委	双击 Windows 免安装 exe，pywebview + WebView2 原生窗口，业务代码热更新	1 分钟

> **最短验证路径（不需要装任何东西，也不需要 API Key）：**在在线演示或本地应用里点侧栏「一键加载演示数据（6 篇）」，然后进「跨文献问答与溯源」，问「训练用了什么 GPU？数量和型号？」，答案会带可点击的来源角标，点角标即可跳到原文页码与段落。

（五）一次完整的使用流程

以「读文献 → 问 → 溯源 → 写」为主线，串起六个模块。

第一步，传文献：上传 PDF（单篇或批量）。系统自动完成细粒度解析：标题树、表格文本化、插图与图注关联、LaTeX 公式还原、References 抽取；扫描件自动识别并走 OCR。损坏、截断、加密的 PDF 在入库前拦截，不留「幽灵文献」（_validate_prepared@services/library.py:597）。

第二步，问：在「跨文献问答与溯源」提问，或让「智能总助手」用自然语言下指令。系统做查询扩展（中文问 → 英文检索词）、Dense + Sparse 混合检索、RRF 融合、来源截断与版本仲裁，最后由拒答闸门判定是否值得调用模型。

第三步，溯源：答案里每条事实带【n】角标，点角标出溯源卡，可直接跳到阅读页的原文页码与段落。

第四步，读：进「版式阅读与智能翻译」，双栏论文按原版式渲染、左右栏不串；中英对照可同步滚动，划词即译。

第五步，追与图：「论文检索与追踪」按主题做增量追踪；「引用图谱与综述」基于真实引用边建图，输出社区结构、基石论文与 frontier，并据此生成综述与 Future Work。

第六步，写：「写作辅助与可视化」生成提纲与正文，References 由系统确定性拼接，CSV 导入后自动出图，数值事实直接注入 Caption。

（六）部署与运行方式

六种运行方式共用同一套业务代码，差异只在入口与部署形态。

表 1-4　六种部署与运行方式

方式	命令 / 入口	说明
本地开发	conda activate ai_research → streamlit run app.py	5 家模型平台走同一套 OpenAI 兼容协议，切换不改业务代码（PROVIDERS@core/providers.py:33）
Windows 一键	双击 start.bat	自动定位 conda 环境、装依赖、起服务、开浏览器

方式	命令 / 入口	说明
macOS / Linux 一键	bash start.sh	同上
Docker	docker compose up -d	健康检查通过后 Up (healthy)；卷属主不对时先 chown -R 10001:10001 ./storage ./logs
桌面版	启动桌面版.vbs（开发态）或打包 exe	业务代码放在 app_bundle/，首次启动复制到 %APPDATA% 后支持热更新
在线	Streamlit Community Cloud	仓库 Public，streamlit_app.py 为入口，密钥走 Secrets

（七）已知边界（先说在前面）

诚实声明比被问出来更好。以下是目前没有覆盖或明确受限的地方。

- 未覆盖赛题「20 篇批量文献」规模。本地语料按 arXiv 号合并后只有 2 - 3 个逻辑节点、1 条真实引用边，图谱指标在小语料下仅供参考，我们没有宣称支持 20 篇。
- 纯中文句的引用一致性校验盲区。锚点只取英文词与数字，锚点少于 2 个时跳过判定（_anchor_tokens@core/guardrails.py:51）。
- 并发写库无文件锁。容器每会话一份，但写库是整体覆盖，短期待遇并发上传有丢块风险。
- 扫描件依赖带视觉能力的平台。内置 5 家里深度求索与本地离线模型没有视觉能力，换到这两家时扫描件是直接拒收入库（scanned_no_ocr），不是「降级但还能用」。
- 中文向量偏弱。同一篇论文改名的整篇相似度 0.891292 够不上 0.90 阈值，靠分段指纹兜底（drop_stale_versions@services/rag.py:91 的版本仲裁同理）。
- 沙箱为软隔离，非操作系统级强隔离。

二、项目亮点

下列六项是本项目与其他同类工具拉开差距的地方，每项均给出可复跑的实测数据。

（一）抗幻觉四道防线：把「答错能不能被抓出来」做成代码判定

表 2-1　四道防线的机制与实测结果

防线	机制	实测结果
生成前 · 拒答闸门	最高相关度低于阈值 0.35 直接拒答，一次模型调用都不发（RELEVANCE_FLOOR@services/rag.py:46）	无关问题最高相关度 0.1820，0 次 LLM 调用；真实拒答文案：「根据提供的文献资料，无法确定……」
生成时 · 角标约束	系统提示强制每条事实带来源角标	细粒度三问全命中真值：Adam β1/β2 = 0.9/0.98、8×P100、BLEU 28.4/41.8，溯源 6/6 带页码
生成后 · 逐句校验	逐句算锚点覆盖率，低于阈值判可疑并触发重生成（check_citation_support@core/guardrails.py:96）	真实句覆盖率 84.5% 通过；编造句覆盖率 10% 被标记
收尾 · 确定性拼接	参考文献由系统从 PDF References 段真实抽取后拼出，模型没有机会碰（build_references_section@services/writing.py:195）	40 条参考文献逐字一致 40/40、占位串 0 处、14364 字、44.6s

为什么这是关键差异：多数系统只能展示「答得好」；MoreHand 能展示「答错会被当场抓住」。前者是宣传语，后者是可复跑的代码判定，评审可以用一条命令验证。

（二）先保住版式，再谈检索

- 双栏几何判定要求 4 条必要条件同时成立（横跨否决、空白带采样、栏宽平衡、两侧块数），任一不满足就保守回退单栏（_detect_two_columns@services/parsing.py:196）。实测：单栏语料 1111 页误判率 19.1% → 0%，真双栏 BERT 正文页 10/10 正确识别。
- 长文档分批工程化：541 页官方教材（22MB）→ 537 页提取 / 1305 个文本块 / 25.32s / 峰值内存 893.1MB / 0 失败页，进度回调 11 次。
- 扫描件走视觉模型：前 3 页原生文本 0 字符即判定 detect_scanned=True，单页 OCR 2833 字符 / 9.3s，词级准确率 93.9% / 召回 98.5%。
- 内容级语义去重不依赖文件名：同篇论文 v1↔v7 相似度 0.963228（阈值 0.90），完全改名后仍命中 1.000000。

（三）意图驱动的 Agent 内核，不是聊天框

- 22 个 Function Schema 全部接入 6 专家路由白名单，无孤岛工具（build_registry@tools/definitions.py:379）。
- 一次自然语言指令 → 规划 → 路由 → 多轮工具调用 → 结果，每一步都写进 Trace，可逐跳复盘。
- 沙箱内执行（AST 白名单 + 资源限额），提示注入检测与密钥脱敏在同一条链路上（check_injection@core/guardrails.py:121、redact_secrets@core/guardrails.py:130）。

（四）引用图谱挖掘与图谱驱动的综述

用真实引用边建图（networkx 真实计算，不是模型编）：PageRank、介数中心性、特征向量中心性（services/graph.py:276）、社区发现、frontier 五

维打分，并在此之上做图谱驱动综述与 Future Work 可达性过滤。实测：综述 1928 字且包含真实年份 2017/2018（没有退化成套话）；Future Work 抽取 2 条、状态 ['存疑','已解决'] 有效状态 2/2；跨文献对比 918 字双向实体均命中。

（五）真实外部数据源与统一降级矩阵

- 被引数、DOI、期刊信息取自 OpenAlex / Semantic Scholar / Crossref / GitHub 等真实源，5 篇知名论文的被引数与 OpenAlex 真值一致（如 AlexNet 108512）；取不到时明说 title_attribution_unconfirmed=True，不猜测、不给好看的错值。
- 降级矩阵覆盖 LLM 不可用、外网中断、视觉能力缺失三类故障，失败时显式标注「以下内容不是模型生成」，不静默失败；无 Key 时可跑通 PDF 原生解析、混合检索、语义去重、引用图谱、追踪增量比对照常运行。

（六）可复现、可审计：每个数字都有复跑命令

表 2-2　关键事实、实测值与复跑命令

事实	实测值	复跑命令
单元测试	238/238 通过	python -m unittest discover -s tests
UI 冒烟	9 个页面 + 9 项故障注入，0 未捕获异常	python scripts/_smoke_ui.py
阅读页专项	62/62 通过	python scripts/_smoke_read.py
真实 API 链路	21/21 通过	python scripts/_verify_live_api.py
技术文档代码引用	236 条引用，0 疑似错误、0 无法判定	python scripts/_check_doc_refs.py
官方用例端到端	双栏判定、541 页入库、扫描件 OCR、被引数校验	python scripts/verify_official.py

> **注意：**verify_official.py / bench_graph.py / selftest.py 都会清空

文献库，库非空时直接拒绝运行，备份后加 --force 才执行。

三、AI 测试技术落地使用情况概述

测试分三层：单元测试层保证函数级正确，界面与交互层保证「看起来正常」也真的正确，真实大模型端到端层保证整条链路在真实模型下成立。

（一）三层测试体系

表 3-1　三层测试体系与实测结果

层级	规模	结果	复跑命令
单元测试层	238 个用例，覆盖 core/services/tools/ui	238/238 通过	python -m unittest discover -s tests
界面与交互层	9 个页面渲染 + 9 项故障注入 + 11 个交互用例（52 条断言）	0 未捕获异常，全部通过	python scripts/_smoke_ui.py、python scripts/_apptest_ui_interact.py
真实大模型端到端层	21 项链路级验收，需真实 Key 与外网	21/21 通过	python scripts/_verify_live_api.py

关键取舍：只做「渲染没报错」的冒烟是不够的，界面看起来正常不等于交互正确。因此交互层用 AppTest 模拟真实点击（跳转落点、清除语义、空状态、指标条、解析后台化），并对每条断言做反向对照（故意注入缺陷，验证门禁能翻红）。

（二）真实大模型下的细粒度问答实测

在真实 LLM 下对论文原文提问，答案与真值逐项核对。

表 3-2　真实 LLM 下对论文原文提问的逐项核对

提问	期望真值	实测
Adam 的 β1、β2 是多少？	0.9 / 0.98	命中
用了什么 GPU？数量与型号？	8 × P100	命中

提问	期望真值	实测
BLEU 分数分别是多少？	28.4 / 41.8	命中
溯源是否带页码？	6/6	6/6 带页码

（三）抗幻觉能力的实测证据

表 3-3　五项抗幻觉能力的实测方法与结果

测试	方法	结果
拒答闸门	问库内毫无依据的问题（作者是否在 2031 年获诺贝尔物理学奖）	系统回答「根据提供的文献资料，无法确定……」，0 次 LLM 调用（闸门在生成前就拦下）
逐句引用校验	真实句 vs 编造句分别过覆盖率判定	真实句 84.5% 通过；编造句 10% 被标记
References 白名单	与 PDF References 段逐字比对	40/40 逐字一致，占位串 0 处、14364 字
越界角标	故意让模型引用不存在于白名单的文献	角标越界项被剔除，不进入 References
图谱综述真实性	检查综述是否含真实年份	1928 字综述含 2017/2018，未退化为套话

（四）版式感知与多模态的实测证据

表 3-4　版式感知与多模态能力的实测结果

能力	实测
双栏判定	单栏语料 1111 页误判 0%；真双栏 BERT 正文页 10/10
长文档工程化	541 页 → 537 页提取 / 1305 块 / 25.32s / 893.1MB / 0 失败页
扫描件 OCR	词级准确率 93.9%、召回 98.5%（单页 2833 字符 / 9.3s）
引用锚点	引用与参考文献双向跳转 36/36 锚点可用、死链 0
语义去重	同篇改名后仍命中，相似度 1.000000

（五）外部数据源与降级链路实测

- 真实被引数：5 篇论文与 OpenAlex 真值一致（AlexNet 108512 等）。
- 限流降级：OpenAlex / Semantic Scholar 处于 IP 级 429 时，链路验证的是诚实降级（「取不到就明说」）而非真值获取，这一点我们不粉饰。
- 无 Key 离线：PDF 原生解析、混合检索、语义去重、引用图谱、追踪增量比对照常运行，生成类内容显式标注「以下内容不是模型生成」。
- 断网演练：走统一降级矩阵，界面明示当前处于降级态，不静默失败。

（六）赛题五大能力自评对照

表 3-5　赛题五大能力类别的落地形态与可核验证据

赛题能力大类	本项目的落地形态	可核验的证据
① 多模态与细粒度文档感知	双栏按栏切分、扫描件多模态 OCR、插图与图注关联、图表内容解读、标题树、表格抽取、541 页长文档分批、损坏 PDF 拦截、内容级语义去重、版本差异、双栏原生渲染	1111 页 0% 误判、10/10 双栏、93.9%/98.5% OCR、0.963228/1.000000 去重
② 抗幻觉 RAG	混合检索（Dense+Sparse+RRF+可选 LLM 重排）、拒答闸门、逐句引用一致性校验、修复环、References 白名单、可点击出处锚点、段落级坐标、多篇跨文献对比、注入检测与密钥脱敏	0 次 LLM 拒答、84.5% vs 10%、40/40 逐字一致、6/6 带页码、36/36 锚点
③ 自主工具调用（Function Calling）	22 个工具 Schema、6 专家路由白名单、ReAct 多轮闭环、AST 沙箱、arXiv 定时增量追踪、Trace 可视化	22 工具全部接入白名单、无孤岛；1 分钟内增量追踪实测通过
④ 图谱挖掘	真实引用建图、PageRank/介数/特征向量、社区发现、frontier 五维打分、图谱驱动综述、Future Work 可达性过滤	综述 1928 字含真实年份、Future Work 2 条、对比 918 字双向命中
⑤ 学术 Copilot	无幻觉 References 生成、	40/40 白名单一致、占位符

赛题能力大类	本项目的落地形态	可核验的证据
	TikZ/Graphviz 拓扑图、CSV 自动出图 + 数值事实注入 Caption、中英对照同步阅读	0 处、44.6s

（七）复跑命令清单（评审可直接执行）

下列命令在仓库根目录执行，全部为纯标准库加项目内脚本，不需要额外环境。

```
PY=C:/Users/Administrator/anaconda3/envs/ai_research/python.exe

$PY -m unittest discover -s tests # 期望 238 OK

$PY scripts/_smoke_ui.py # 期望 9 页 0 异常 + 9 故障注入

$PY scripts/_smoke_read.py # 期望 62/62

$PY scripts/_apptest_ui_interact.py # 期望 11 用例 52 断言

$PY scripts/_verify_live_api.py # 期望 21/21（需真实 Key 与外网）

$PY scripts/_check_doc_refs.py # 期望 236 条 0 疑似 0 无法判定
```

四、项目涉及的关键技术

下表先给出技术全景，随后逐条展开检索增强、防幻觉、智能体内核、图谱与 Copilot 五条主链路的工程实现。

（一）技术全景

表 4-1　八个技术领域的采用方案、选型理由与代码落点

技术领域	采用方案	选型理由	代码落点
文档解析	PyMuPDF + 版式几何判定 + 扫描件视觉 OCR	几何判定不依赖模型猜版面，成本低、可解释；OCR	services/parsing.py

技术领域	采用方案	选型理由	代码落点
		只在无文本层时启用	
向量检索	ChromaDB + ONNX MiniLM-L6-v2（384 维）+ BM25Plus + RRF 融合	稠密管语义、稀疏管术语，RRF 融合避免单一召回面偏科	services/rag.py、services/state.py
生成与防幻觉	Prompt 约束 + 拒答闸门 + 逐句引用校验 + 修复环 + 确定性 References 拼接	把「不 hallucinate」变成可测的工程判定，而不是提示词里的祈使句	core/guardrails.py、services/rag.py、services/writing.py
智能体	Function Calling（22 Schema）+ 6 专家路由 + 规划/记忆/沙箱/Trace	工具白名单杜绝幻觉调用；Trace 让每一步可复盘	core/agent.py、core/router.py、tools/definitions.py
图谱	networkx（PageRank/介数/特征向量）+ 社区发现 + frontier 五维	真实计算而非模型编造，指标可核对	services/graph.py
前端	Streamlit 1.63 + 自研设计令牌层（唯一强调色 #1f5fae）	单一令牌源保证视觉一致；文档面保留衬线与荧光笔语义色	ui/design.py、ui/ 各页面模块
桌面端	pywebview + WebView2 + 业务代码热更新（.build_id 比对）	免安装、免 Python 环境；重打包后自动换新业务代码	desktop/
工程化	Docker 一键部署、5 家平台 OpenAI 兼容抽象、后台任务线程池、逐跳 Trace	降级可预期、故障可定位	core/providers.py、services/tasks.py、core/trace.py

（二）检索增强：混合检索与 RRF 融合

一次提问的完整链路如下。

第一，查询扩展：中文提问转英文检索词、同义词与缩写变体（LLM 不可用时回退内置词表）。

第二，双路召回：Dense（DENSE_K=12，变体上限 5）与 Sparse（BM25Plus，SPARSE_K=12，中文 bigram）；强命中（相似度不低于 0.55）时跳过扩展。

第三，融合与收敛：RRF 融合（rrf_fuse@services/rag.py:258，融合池 24 条）→ 来源截断（cap_by_source@services/rag.py:267，防大书淹没检索）→ 版本仲裁去重（drop_stale_versions@services/rag.py:91）→ 收敛到 FINAL_K=6。

第四，可选 LLM 重排：失败时静默回退到融合序，不阻断。

第五，段落级标注：为每个片段补上「[n] 来源：《X》第 N 页第 M 段」坐标（annotate_paragraphs → build_context@services/rag.py:709）。

（三）防幻觉：四道防线的工程实现

- 拒答闸门：闸门设在生成之前，相关度不达标直接返回拒答文案，既不浪费模型调用，也不给模型「编一个」的机会。
- 逐句引用校验：把答案拆句，逐句算与来源片段的锚点覆盖率（_anchor_tokens@core/guardrails.py:51 只取英文词与数字，避免中文句误判），不达阈值触发一次重生成（MAX_REPAIR=1 防死循环）。
- References 白名单：写作侧对角标做越界与条目实质匹配双重校验（validate_citations@core/guardrails.py:140），模型产出与白名单冲突时以白名单为准。
- 安全链路：提示注入检测（check_injection@core/guardrails.py:121）与密钥脱敏（redact_secrets@core/guardrails.py:130）在同一条护栏链上。

（四）智能体内核与工具注册

22 个工具按能力域注册到 6 位专家的白名单上（build_registry@tools/definitions.py:379），每个工具都有入参 schema 与用途描述；路由器把自然语言意图映射到专家，专家再决定调用哪些工具。执行在 AST 白名单沙箱内，Prompt 注入无法越权；每次工具调用都进 Trace，可展开看输入、输出与耗时。

（五）引用图谱挖掘

真实引用边 → networkx 计算 PageRank、介数中心性、特征向量中心性 → 按「被引 + PageRank」混合排序取前 30% 为基石论文 → 社区发现给出研究簇 → frontier 用五维打分（低被引但高相关度/近期性等）挑出前沿方向 → 在此之上生成综述与 Future Work 候选，并做可达性过滤，避免推荐「尚未被引用但也不相关」的论文。

（六）学术 Copilot

- 提纲与正文：分步生成，每步可回看。
- 拓扑图：TikZ / Graphviz 双通道，缺 LaTeX 编译器时展示源码与编辑器链接（不假装编译成功）。
- CSV 自动出图：从表格自动选图型（折线/柱状/散点/相关热力/箱线/雷达），并把真实数值事实注入 Caption，避免「图是图、话是话」。
- 翻译：中英互译 + 中英对照同步滚动 + 划词翻译 + 原位译文叠印四种形态。

（七）工程与交付

- 可复现构建：一键脚本、Docker、桌面版三条路径产出一致的业务代码。

- 可观测：逐跳 Trace + 后台任务进度回调（services/tasks.py 线程池，MAX_PARALLEL=2），耗时 10–30 秒的模型调用一律后台化，不冻结界面。
- 降级不静默：统一的降级矩阵在 UI 上明示当前状态。
- 文档即交付：技术文档 30 页、演示与预指导脚本、评委 FAQ、部署手册等材料与代码同源，技术文档中的每条代码引用都可一键复跑校验。

五、项目相关的图片/照片

以下均为本项目真实运行界面截图（本地运行，演示数据 6 篇，非示意图、无后期拼图），可用于核实界面与功能。

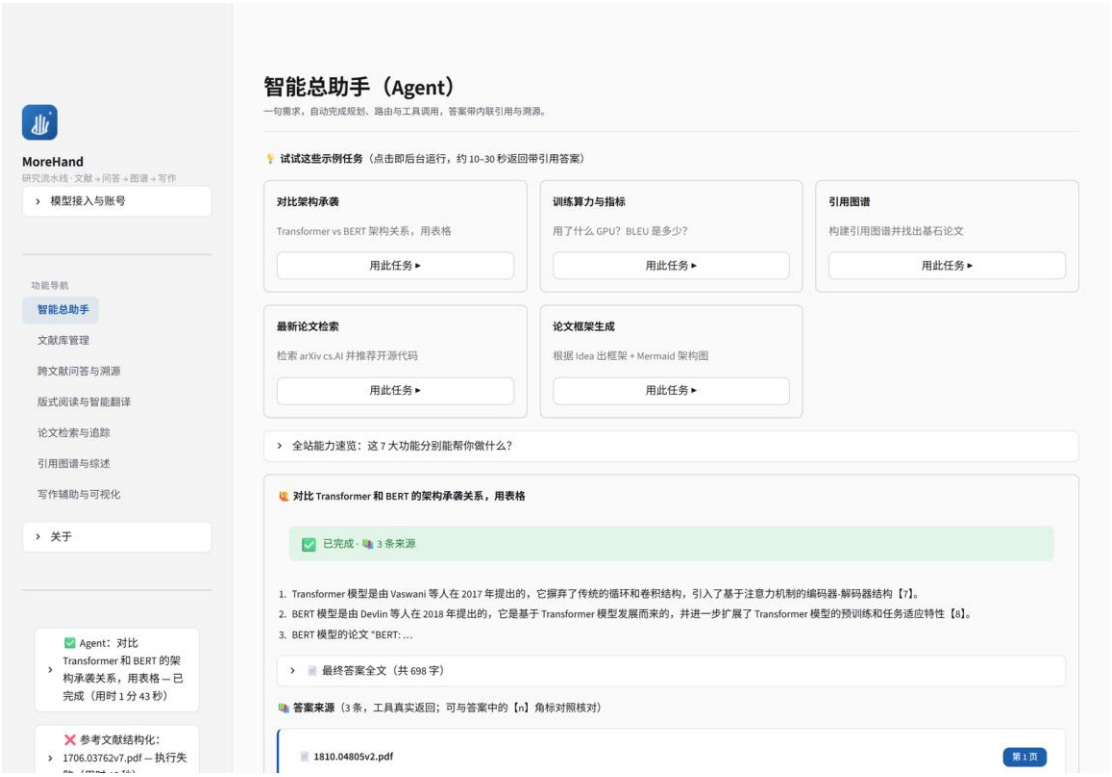


图 1　智能总助手：意图驱动的入口。左侧为九项导航与文献库状态；主区域是自然语言指令入口，下挂工具调用 Trace，用于展示「一句话 → 规划 → 工具调用 → 结果」的完整闭环。

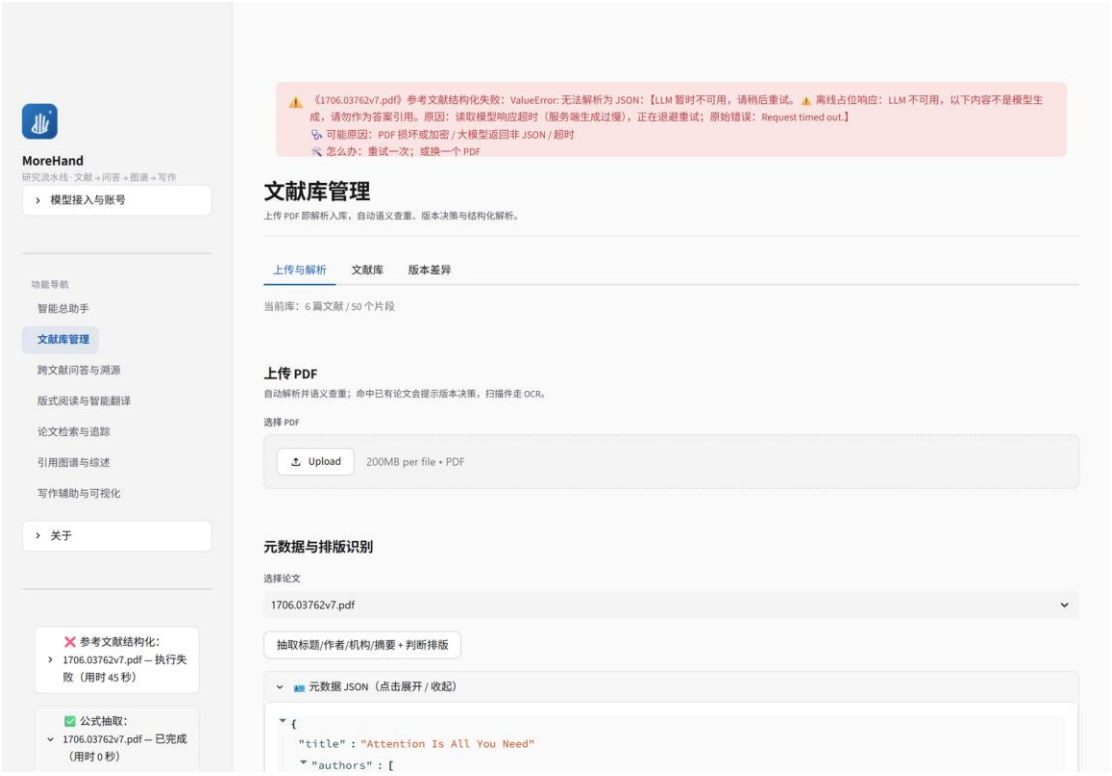


图 2　文献库管理：细粒度解析产物。每篇文献展示解析后的标题树、表格数、插图数、公式数、参考文献数与文本块数，这些是后续检索与溯源的原料。



图 3　跨文献问答与溯源：角标、拒答与调试区。顶部是常用测试问题胶囊与检索范围选择；

中部是提问框与「提问」按钮；下方示例是一次真实拒答：「未在文献库中找到可靠依据，为避免幻觉我宁可不作答」，并显示最高相关度 0.49 低于阈值 0.35、共 6 个候选片段。

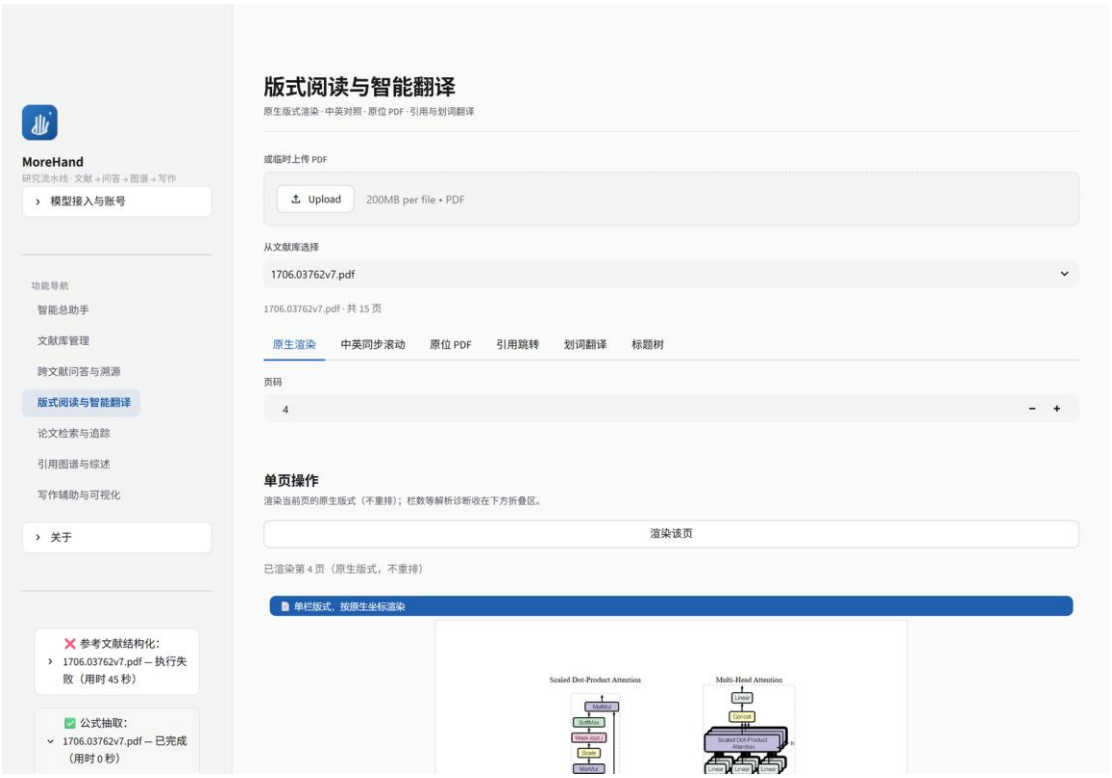


图 4　版式阅读与智能翻译：双栏不串栏。左侧为论文页缩略与工具条，右侧为按原版式渲染的正文，双栏论文的左右栏保持分栏不串；另有中英对照、划词翻译与栏数检测入口。

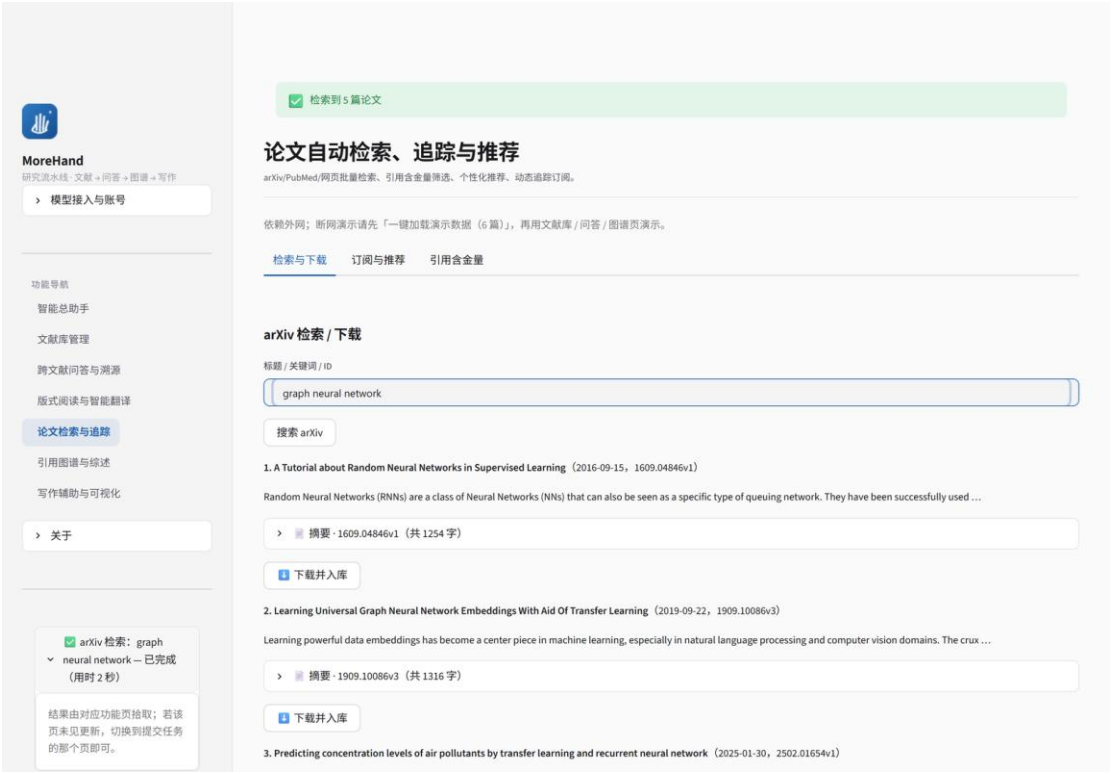


图 5　论文检索与追踪：多源检索与引用含金量。上方为 arXiv / PubMed 多源检索与结果列表（含引用含金量标注），下方为主题订阅与增量追踪面板。

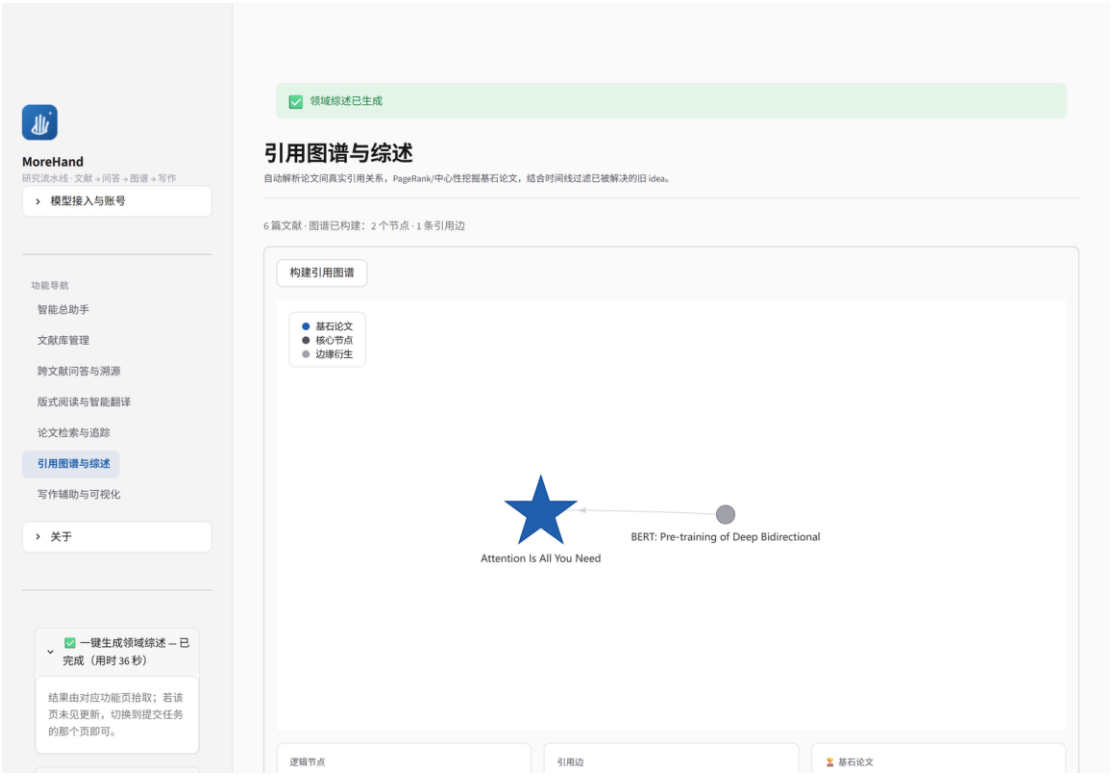


图 6　引用图谱与综述：真实引用边建图。图谱由 PDF 真实引用边构建（networkx 计算），并

给出社区结构、基石论文与图谱明细；右侧可一键生成领域综述与 Future Work。

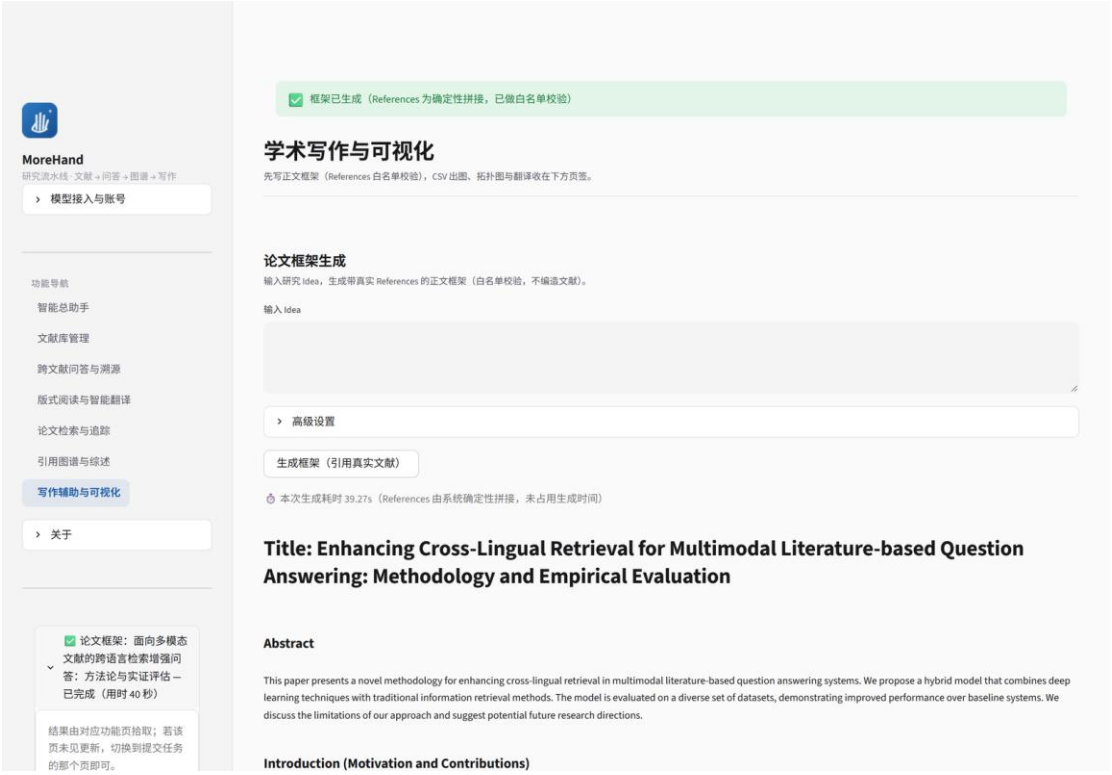


图 7　写作辅助与可视化：References 与自动出图。左下是提纲与正文生成区（References 由系统确定性拼接），右侧是 CSV 导入后自动出图与 Caption 生成。

六、项目之前是否发表过或获奖

如实说明：截至本文档定稿日（2026-10-26），本作品为本次赛事的原创参赛作品，未获得任何奖项，未在学术会议或期刊发表，未申请软件著作权或专利。

具体说明如下。

- 获奖情况：无。截至定稿日，本作品尚未参加任何以「获奖」为结果的评审或赛事，本次参赛是首次公开亮相。
- 发表情况：无。本作品未以论文形式在学术会议或期刊发表，也未预印本公开。
- 知识产权登记：未申请软件著作权、未申请专利。

- 公开时间：代码仓库于 2026 年内首次公开提交（Public），git 历史自 2026 年初立项起连续可查，可作为原创性的佐证。
- 第三方组件许可：本作品使用了一批宽松许可的开源组件（Streamlit、ChromaDB、networkx、PyMuPDF、ONNX Runtime、Transformers、pywebview 等），其许可均为 Apache-2.0 / MIT / BSD 等商业友好类型；这些组件是工具依赖，不构成本作品的原创内容。本作品的原创部分是自研的 Agent 内核、版式解析层、四道防幻觉防线、引用图谱与 Copilot 逻辑。
- 后续声明：若本作品在后续赛事中获奖或被发表，本文档将同步更新该节，并在引用本项目时标注成果来源与获奖信息。

七、若项目是在其他项目基础上优化

如实说明：本作品不是在任何他人已发布或已获奖的项目基础上改造而来，而是由本团队从零自研。以下说明三点。

（一）自研演进路径（可由 git 历史逐条核验）

表 7-1　自研演进路径与各阶段主要产出

阶段	目标	主要产出
立项选题（2026 年初）	明确 AI4S 三大痛点：文献解析串读、LLM 幻觉、离线不可用	需求清单与竞品对比表
原型验证（2026 年上半年）	打通 Streamlit + 基础 RAG 链路	v1 单体原型（已归档至 version_history/）
核心模块开发（2026 年上半年 - 年中）	七大模块并行开发	模块化重构为 core/、services/、tools/、ui/ 分层
抗幻觉专项（2026 年中 - 下半年）	拒答闸门、引用一致性校验、修复环、白名单四道防线	services/rag.py、core/guardrails.py 及配套测试
性能与稳定性（2026 年下	541 页长文档分批入库、单	实测 537 页提取 / 1305

阶段	目标	主要产出
半年）	页超时、内存治理	块 / 25.32s / 893.1MB / 0 失败页
交付与答辩准备（2026 年下半年 - 冬）	技术文档、演示视频、在线部署、团队信息	30 页技术文档、演示与预指导脚本、在线演示地址

优化的是自己的上一版，不是别人的项目：例如把 v1 单体原型重构为分层架构、把「引用校验只查角标范围」升级为「逐句覆盖率判定」、把「拒答写在 prompt 里」升级为「生成前的代码闸门」、把「双栏靠模型猜」升级为「4 条几何必要条件」，这些都是本项目自身版本的迭代，git 提交可逐条追溯。

（二）与通用工具的定位差异（说明我们不是在改造它们）

表 7-2 MoreHand 与四类通用工具的定位差异

工具	它的重心	MoreHand 的重心	关系
Zotero	文献管理（管）	文献理解（懂）：内容级语义去重、版本差异、引用链挖掘	并列工具，非改造对象
Elicit / Semantic Scholar	检索与综述（找得准）	检索之后能不能验真：可点击页码锚点 + 逐句引用校验	并列工具，非改造对象
ChatGPT 等通用模型	通用知识问答	限定在「我给的文献」内作答，不确定就拒答	并列工具，非改造对象
draw.io / Overleaf	绘图与排版	从文献到初稿的闭环，图表数值与正文事实绑定	并列工具，非改造对象

（三）若评审要求更多佐证

可提供：① 连续 git 提交历史（自 2026 年初立项起）；② version_history/ 下的 v1 原型归档；③ 每一项关键能力对应的复跑脚本与实测日志。我们不引用任何未经验证的「既有成果」来抬高本作品。

附录 A · 配套材料索引

随本作品一并交付的材料清单如下，均与代码同源、可逐条核验。

表 A-1　随本作品交付的配套材料

材料	路径	用途
技术文档（30 页）	资料/技术文档.pdf	架构 / 选型 / 参数 / 降级矩阵 / 236 条代码引用
预指导环节介绍	资料/预指导环节介绍.md	面向专家的说明底稿（含追问口径）
预指导视频脚本	资料/预指导视频脚本.md	3 分 30 秒录屏脚本 + 速查卡
演示视频脚本	资料/演示视频脚本.md	决赛 8 分钟完整录屏
演示彩排脚本	资料/演示彩排脚本.md	现场真人操作 7 分 30 秒
评委 FAQ	资料/评委 FAQ.md	22 条高频问题，每条附文件行号依据
5 分钟快速体验	资料/5 分钟快速体验指南.md	评委视角的上手路径
部署操作手册	资料/部署操作手册.md	五步部署 + 报错速查
网络降级演练	资料/网络降级演练手册.md	断网/限流时的降级处置
团队信息	资料/团队信息.md	队伍与分工矩阵

附录 B · 版本与复核

本附录记录本文档的版本与复核基线，供评审核对。

表 B-1　文档版本与复核基线

项	内容
文档版本	v1.0（2026-10-26）
引用校验	scripts/_check_doc_refs.py：技术文档 236 条 / 0 疑似 / 0 无法判定
门禁基线	单元测试 238/238 • UI 冒烟 9 页 0 异常 • 阅读页 62/62 • 真实 API 21/21

项	内容
写作纪律	无实测支撑的功能与数字不写入；界面文案与代码现查现用；旧审计快照中的完成度百分比不作为当前状态引用